Lab 9 — Source Code Report

InvoiceDoc2: Discount & Configuration Features

Generated: 5/6/2026, 11:22:23 AM

Files Modified / Created in Lab 9:

1. [NEW]   Step 1   database/sql/004_invoice_lab3_delta.sql
2. [UPDATED] Step 2   server/src/services/salesPersons.service.js
3. [UPDATED] Step 2   server/src/controllers/salesPersons.controller.js
4. [UPDATED] Step 2   server/src/routes/salesPersons.routes.js
5. [NEW]   Step 3   server/src/services/configuration.service.js
6. [NEW]   Step 3   server/src/controllers/configuration.controller.js
7. [NEW]   Step 3   server/src/routes/configuration.routes.js
8. [UPDATED] Step 3   server/src/app.js
9. [UPDATED] Step 4   server/src/services/invoices.service.js
10. [NEW]   Step 5   client/src/api/configuration.api.js
11. [UPDATED] Step 6   client/src/api/salesPersons.api.js
12. [NEW]   Step 7   client/src/pages/salesPersons/SalesPersonList.jsx
13. [NEW]   Step 7   client/src/pages/salesPersons/SalesPersonPage.jsx
14. [UPDATED] Step 7   client/src/main.jsx
15. [UPDATED] Step 8   client/src/components/LineItemsEditor.jsx
16. [UPDATED] Step 8   client/src/components/LineItemRow.jsx
17. [UPDATED] Step 9   client/src/components/InvoiceForm.jsx
18. [UPDATED] Step 10 client/src/pages/invoices/InvoicePage.jsx

DB Delta Script

```
\set ON_ERROR_STOP on

-- Add discount columns to invoice_line_item
ALTER TABLE invoice_line_item
  ADD COLUMN IF NOT EXISTS line_discount_percent numeric(5,2) NOT NULL DEFAULT 0.00,
  ADD COLUMN IF NOT EXISTS line_discount_amount numeric(14,2) NOT NULL DEFAULT 0.00,
  ADD COLUMN IF NOT EXISTS line_net_price numeric(14,2) NOT NULL DEFAULT 0.00;

-- Backfill: set line_net_price = extended_price where no discount
UPDATE invoice_line_item
  SET line_net_price = extended_price
  WHERE line_discount_amount = 0;

-- Add totals columns to invoice
ALTER TABLE invoice
  ADD COLUMN IF NOT EXISTS vat_rate numeric(5,4) NOT NULL DEFAULT 0.07,
  ADD COLUMN IF NOT EXISTS total_price numeric(14,2),
  ADD COLUMN IF NOT EXISTS total_discount numeric(14,2) NOT NULL DEFAULT 0.00,
  ADD COLUMN IF NOT EXISTS vat_amount numeric(14,2);

-- Backfill: total_price = total_amount, vat_amount = vat where NULL
UPDATE invoice SET total_price = total_amount WHERE total_price IS NULL;
UPDATE invoice SET vat_amount = vat WHERE vat_amount IS NULL;

-- Create configuration table
CREATE TABLE IF NOT EXISTS configuration (
  id bigint PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  key text UNIQUE NOT NULL,
  value text NOT NULL
);

-- Seed vat_percent
INSERT INTO configuration (key, value) VALUES ('vat_percent', '7') ON CONFLICT DO NOTHING;

```

Sales Person Service (CRUD)

```
import { pool } from "../db/pool.js";

export async function listSalesPersons({ search, page, limit }) {
  const offset = (Number(page) - 1) * Number(limit);
  const searchParam = `%${search}%`;

  const countResult = await pool.query(
    `SELECT COUNT(*) as total FROM sales_person
     WHERE code ILIKE $1 OR name ILIKE $1`,
    [searchParam],
  );

  const { rows } = await pool.query(
    `SELECT id, code, name, start_work_date
     FROM sales_person
     WHERE code ILIKE $1 OR name ILIKE $1
     ORDER BY code ASC
     LIMIT $2 OFFSET $3`,
    [searchParam, Number(limit), offset],
  );

  const total = Number(countResult.rows[0].total);
  return {
    data: rows,
    total,
    page: Number(page),
    limit: Number(limit),
    totalPages: Math.ceil(total / Number(limit)),
  };
}

export async function getSalesPerson(code) {
  const { rows } = await pool.query(
    "SELECT id, code, name, start_work_date FROM sales_person WHERE code = $1",
    [code],
  );
  return rows[0] || null;
}

export async function createSalesPerson({ code, name, start_work_date }) {
  let resolvedCode = code;
  if (!resolvedCode || String(resolvedCode).trim() === "") {
    const maxRes = await pool.query("SELECT MAX(id) as m FROM sales_person");
    const nextId = (Number(maxRes.rows[0].m) || 0) + 1;
    resolvedCode = `SP${nextId.toString().padStart(3, "0")}`;
  }
  const { rows } = await pool.query(
    `INSERT INTO sales_person (id, created_at, code, name, start_work_date)
     VALUES ((SELECT coalesce(max(id),0)+1 FROM sales_person), now(), $1, $2, $3)
     RETURNING id, code, name, start_work_date`,
    [resolvedCode, name, start_work_date || null],
  );
  return rows[0];
}

export async function updateSalesPerson(code, { name, start_work_date }) {
  const { rows } = await pool.query(
    `UPDATE sales_person SET name=$1, start_work_date=$2 WHERE code=$3
     RETURNING id, code, name, start_work_date`,
    [name, start_work_date || null, code],
  );
  return rows[0] || null;
}

export async function deleteSalesPerson(code) {
  await pool.query("DELETE FROM sales_person WHERE code=$1", [code]);
  return { ok: true };
}

```

Sales Person Controller

```
import { listSalesPersons, getSalesPerson, createSalesPerson, updateSalesPerson, deleteSalesPerson } from "../services/salesPersons.service.js";

export async function handleList(req, res) {
  try {
    const { search = "", page = 1, limit = 10 } = req.query;
    const result = await listSalesPersons({ search, page, limit });
    res.json({
      success: true,
      data: result.data,
      meta: {
        total: result.total,
        page: result.page,
        limit: result.limit,
        totalPages: result.totalPages,
      },
    });
  } catch (err) {
    res.status(500).json({ success: false, error: { message: err.message } });
  }
}

export async function handleGet(req, res) {
  try {
    const { code } = req.params;
    const result = await getSalesPerson(code);
    if (!result) return res.status(404).json({ success: false, error: { message: `Sales person not found: ${code}` } });
    res.json({ success: true, data: result });
  } catch (err) {
    res.status(500).json({ success: false, error: { message: err.message } });
  }
}

export async function handleCreate(req, res) {
  try {
    const { code, name, start_work_date } = req.body;
    const result = await createSalesPerson({ code, name, start_work_date });
    res.status(201).json({ success: true, data: result });
  } catch (err) {
    res.status(500).json({ success: false, error: { message: err.message } });
  }
}

export async function handleUpdate(req, res) {
  try {
    const { code } = req.params;
    const { name, start_work_date } = req.body;
    const result = await updateSalesPerson(code, { name, start_work_date });
    if (!result) return res.status(404).json({ success: false, error: { message: `Sales person not found: ${code}` } });
    res.json({ success: true, data: result });
  } catch (err) {
    res.status(500).json({ success: false, error: { message: err.message } });
  }
}

export async function handleDelete(req, res) {
  try {
    const { code } = req.params;
    await deleteSalesPerson(code);
    res.json({ success: true, data: { ok: true } });
  } catch (err) {
    res.status(500).json({ success: false, error: { message: err.message } });
  }
}

```

Sales Person Routes

```
import { Router } from "express";
import { handleList, handleGet, handleCreate, handleUpdate, handleDelete } from "../controllers/salesPersons.controller.js";

const router = Router();
router.get("/", handleList);
router.get("/:code", handleGet);
router.post("/", handleCreate);
router.put("/:code", handleUpdate);
router.delete("/:code", handleDelete);

export default router;

```

Configuration Service

```
import { pool } from "../db/pool.js";

export async function getConfig(key) {
  const r = await pool.query("SELECT value FROM configuration WHERE key = $1", [key]);
  if (r.rowCount === 0) throw new Error(`Config not found: ${key}`);
  return r.rows[0];
}

```

Configuration Controller

```
import { getConfig } from "../services/configuration.service.js";

export async function handleGetConfig(req, res) {
  try {
    const { key } = req.params;
    const result = await getConfig(key);
    res.json({ success: true, data: result });
  } catch (err) {
    res.status(404).json({ success: false, error: { message: err.message } });
  }
}

```

Configuration Routes

```
import { Router } from "express";
import { handleGetConfig } from "../controllers/configuration.controller.js";

const router = Router();
router.get("/:key", handleGetConfig);

export default router;

```

App Entry (mount /api/config)

```
// Backend entry: receives HTTP requests and forwards to routes (customers, products, invoices, reports)
import express from "express";
import cors from "cors";
import { execSync } from "child_process";
import path from "path";
import fs from "fs";

import logger from "./utils/logger.js";
import invoicesRoutes from "./routes/invoices.routes.js";
import salesPersonsRoutes from "./routes/salesPersons.routes.js";
import reportsRoutes from "./routes/reports.routes.js";
import customersRoutes from "./routes/customers.routes.js";
import productsRoutes from "./routes/products.routes.js";
import configurationRoutes from "./routes/configuration.routes.js";

const app = express();

// ANSI colors for request log (dev only; production uses JSON without codes)
const c = {
  reset: "\x1b[0m",
  dim: "\x1b[2m",
  green: "\x1b[32m",
  yellow: "\x1b[33m",
  blue: "\x1b[34m",
  magenta: "\x1b[35m",
  cyan: "\x1b[36m",
  red: "\x1b[31m",
};
const methodColor = { GET: c.green, POST: c.cyan, PUT: c.yellow, DELETE: c.magenta };
const statusColor = (s) => (s >= 500 ? c.red : s >= 400 ? c.yellow : s >= 300 ? c.blue : c.green);
const useColor = process.env.NODE_ENV !== "production" && process.stdout.isTTY;

// Request logging: one line per request, colored by method and status in dev
app.use((req, res, next) => {
  const start = Date.now();
  const method = req.method;
  res.on("finish", () => {
    const duration = Date.now() - start;
    const status = res.statusCode;
    let msg = `[${method}] ${req.originalUrl} ${status} ${duration}ms`;
    if (useColor) {
      const m = methodColor[method] || c.dim;
      const s = statusColor(status);
      msg = `${m}[${method}]${c.reset} ${req.originalUrl} ${s}${status}${c.reset} ${c.dim}${duration}
ms${c.reset}`;
    }
    logger.info(msg, { method, url: req.originalUrl, status, durationMs: duration });
  });
  next();
});

// CORS: allow origin from CORS_ORIGIN env, or * if unset (e.g. dev). Allows common methods/headers.
const corsOrigin = process.env.CORS_ORIGIN;
app.use(
  cors({
    origin: corsOrigin === undefined || corsOrigin === "" ? true : corsOrigin.split(",").map((o) =>
o.trim()),
    methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
    allowedHeaders: ["Content-Type", "Authorization"],
  })
);
app.use(express.json());

// Health check
app.get("/health", (_, res) => res.json({ ok: true }));

// Check if repo has newer commits (for frontend update banner). Uses GIT_SHA from env, or git rev-parse
when running dev.
const REPO = "llbumpbumpll/InvoiceDoc2";
const REPO_URL = `https://github.com/${REPO}`;
function getCurrentSha() {
  const fromEnv = process.env.GIT_SHA || process.env.VERCEL_GIT_COMMIT_SHA ||
process.env.RENDER_GIT_COMMIT_SHA;
  if (fromEnv) return fromEnv.trim();
  let repoRoot = process.cwd();
  if (!fs.existsSync(path.join(repoRoot, ".git"))) {
    repoRoot = path.resolve(repoRoot, "..");
```

```
  }
  try {
    return execSync("git rev-parse HEAD", { cwd: repoRoot, encoding: "utf8" }).trim();
  } catch {
    return null;
  }
}
app.get("/api/updates-check", async (_, res) => {
  const currentSha = getCurrentSha();
  if (!currentSha) {
    res.setHeader("X-Update-Available", "false");
    return res.json({ updateAvailable: false });
  }
  try {
    const r = await fetch(`https://api.github.com/repos/${REPO}/commits/main`, {
      headers: { Accept: "application/vnd.github.v3+json" },
    });
    if (!r.ok) {
      res.setHeader("X-Update-Available", "false");
      return res.json({ updateAvailable: false });
    }
    const data = await r.json();
    const latestSha = data?.sha;
    const updateAvailable = !!latestSha && latestSha !== currentSha;
    res.setHeader("X-Update-Available", updateAvailable ? "true" : "false");
    if (updateAvailable) res.setHeader("X-Update-Repo-Url", REPO_URL);
    res.json({ updateAvailable, repoUrl: REPO_URL });
  } catch {
    res.setHeader("X-Update-Available", "false");
    res.json({ updateAvailable: false });
  }
});

// API routes
app.use("/api/customers", customersRoutes);
app.use("/api/products", productsRoutes);
app.use("/api/invoices", invoicesRoutes);
app.use("/api/sales-persons", salesPersonsRoutes);
app.use("/api/reports", reportsRoutes);
app.use("/api/config", configurationRoutes);

const port = process.env.PORT || 4000;
const host = process.env.HOST || "0.0.0.0";
app.listen(port, host, () => logger.info(`Invoice server listening on ${host}:${port}`));

```

Invoice Service (discount calcs)

```
// Invoice CRUD: list (search, pagination, sort), get with line items, create/update/delete. Transactions for create/update.
import { pool } from "../db/pool.js";

export async function listInvoices({
  search = "",
  page = 1,
  limit = 10,
  sortBy = "invoice_date",
  sortDir = "desc",
} = {}) {
  const offset = (Number(page) - 1) * Number(limit);

  const allowedSort = ["invoice_no", "customer_name", "invoice_date", "amount_due"];
  const sortColumn = allowedSort.includes(sortBy) ? sortBy : "invoice_date";
  const sortDirection = sortDir === "asc" ? "ASC" : "DESC";

  const searchParam = `%${search}%`;

  const countResult = await pool.query(
    `
      SELECT COUNT(*) as total
      FROM invoice i
      JOIN customer c ON c.id = i.customer_id
      WHERE i.invoice_no ILIKE $1 OR c.name ILIKE $1
    `,
    [searchParam],
  );
  const total = Number(countResult.rows[0].total);

  const { rows } = await pool.query(
    `
      SELECT i.invoice_no, i.invoice_date, i.amount_due,
       c.name as customer_name,
       sp.name as sales_person_name
      FROM invoice i
      JOIN customer c ON c.id = i.customer_id
      LEFT JOIN sales_person sp ON sp.id = i.sales_person_id
      WHERE i.invoice_no ILIKE $1 OR c.name ILIKE $1
      ORDER BY ${sortColumn} ${sortDirection} NULLS LAST, i.id DESC
      LIMIT $2 OFFSET $3
    `,
    [searchParam, Number(limit), offset],
  );

  return {
    data: rows,
    total,
    page: Number(page),
    limit: Number(limit),
    totalPages: Math.ceil(total / Number(limit)),
  };
}

/** Resolve invoice_no to id (for internal use). */
async function resolveInvoiceId(invoice_no) {
  const r = await pool.query("SELECT id FROM invoice WHERE invoice_no = $1", [invoice_no]);
  return r.rowCount > 0 ? r.rows[0].id : null;
}

export async function getInvoice(idOrInvoiceNo) {
  // Support both id (number) and invoice_no (string) for backward compatibility during migration
  let id = idOrInvoiceNo;
  if (typeof idOrInvoiceNo === "string" && String(idOrInvoiceNo).trim() !== "" && isNaN(Number(idOrInvoiceNo))) {
    id = await resolveInvoiceId(String(idOrInvoiceNo).trim());
    if (id == null) return null;
  } else {
    id = Number(idOrInvoiceNo);
  }

  const header = await pool.query(
    `
      select i.invoice_no, i.invoice_date, i.customer_id, i.total_amount, i.vat, i.amount_due,
       i.vat_rate, i.total_price, i.total_discount, i.vat_amount,
       c.code as customer_code, c.name as customer_name,
       c.address_line1, c.address_line2,
```

```
        co.name as country_name,
        sp.code as sales_person_code,
        sp.name as sales_person_name
      from invoice i
      join customer c on c.id = i.customer_id
      left join country co on co.id = c.country_id
      left join sales_person sp on sp.id = i.sales_person_id
      where i.id = $1
    `,
    [id],
  );

  if (header.rowCount === 0) return null;

  const lines = await pool.query(
    `
      select li.id,
             p.code as product_code, p.name as product_name,
             u.code as units_code,
             li.quantity, li.unit_price, li.extended_price,
             li.line_discount_percent, li.line_discount_amount, li.line_net_price
      from invoice_line_item li
      join product p on p.id = li.product_id
      join units u on u.id = p.units_id
      where li.invoice_id = $1
      order by li.id
    `,
    [id],
  );

  return { header: header.rows[0], line_items: lines.rows };
}

/** Resolve product_code to id and compute discount fields. */
async function enrichLineItems(client, line_items) {
  const enriched = [];
  for (const li of line_items) {
    const product_code = li.product_code != null ? String(li.product_code).trim() : null;
    if (!product_code) throw new Error("Line item missing product_code");
    const pr = await client.query(
      "SELECT id, unit_price FROM product WHERE code = $1",
      [product_code],
    );
    if (pr.rowCount === 0) throw new Error(`Product not found: ${product_code}`);
    const product_id = pr.rows[0].id;
    const unit_price = li.unit_price ?? Number(pr.rows[0].unit_price ?? 0);
    const extended_price = Number(li.quantity) * Number(unit_price);
    const line_discount_percent = Number(li.line_discount_percent || 0);
    const line_discount_amount = Math.round(extended_price * line_discount_percent / 100 * 100) / 100;
    const line_net_price = Math.round((extended_price - line_discount_amount) * 100) / 100;
    enriched.push({ ...li, product_id, unit_price, extended_price, line_discount_percent,
line_discount_amount, line_net_price });
  }
  return enriched;
}

export async function createInvoice({ invoice_no, customer_code, sales_person_code, invoice_date,
vat_rate, line_items }) {
  const client = await pool.connect();
  try {
    await client.query("begin");

    const code = customer_code != null ? String(customer_code).trim() : "";
    const cust = await client.query("SELECT id, credit_limit FROM customer WHERE code = $1", [code]);
    if (cust.rowCount === 0) throw new Error(`Customer not found: ${code}`);
    const customer_id = cust.rows[0].id;

    // Resolve sales_person_code !’ id
    let sales_person_id = null;
    if (sales_person_code) {
      const sp = await client.query("SELECT id FROM sales_person WHERE code = $1", [sales_person_code]);
      if (sp.rowCount === 0) throw new Error(`Sales person not found: ${sales_person_code}`);
      sales_person_id = sp.rows[0].id;
    }

    let resolvedInvoiceNo = invoice_no;
    if (!resolvedInvoiceNo || String(resolvedInvoiceNo).trim() === "") {
      const maxRes = await client.query("SELECT MAX(id) as m FROM invoice");
      const nextId = (maxRes.rows[0].m || 0) + 1;
```

```
153       resolvedInvoiceNo = `INV-${nextId.toString().padStart(3, "0")}`;
154     }
155
156     const enriched = await enrichLineItems(client, line_items);
157
158     const total_price = enriched.reduce((s, x) => s + x.extended_price, 0);
159     const total_discount = enriched.reduce((s, x) => s + x.line_discount_amount, 0);
160     const vat_amount = Math.round((total_price - total_discount) * Number(vat_rate) * 100) / 100;
161     const amount_due = (total_price - total_discount) + vat_amount;
162
163     if (cust.rows[0].credit_limit != null) {
164       const limit = Number(cust.rows[0].credit_limit);
165       if (amount_due > limit) {
166         throw new Error(`Amount due (${amount_due}) exceeds customer credit limit (${limit}).`);
167       }
168     }
169
170     const inv = await client.query(
171       `
172         insert into invoice (id, created_at, invoice_no, invoice_date, customer_id, sales_person_id,
173           total_amount, vat, amount_due, vat_rate, total_price, total_discount, vat_amount)
174         values (
175           (select coalesce(max(id),0)+1 from invoice),
176           now(),
177           $1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11
178         )
179         returning id, invoice_no
180       `,
181       [resolvedInvoiceNo, invoice_date, customer_id, sales_person_id,
182        total_price, vat_amount, amount_due, Number(vat_rate), total_price, total_discount, vat_amount],
183     );
184
185     const invoice_id = inv.rows[0].id;
186
187     for (const li of enriched) {
188       await client.query(
189         `
190           insert into invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price,
extended_price,
191             line_discount_percent, line_discount_amount, line_net_price)
192           values (
193             (select coalesce(max(id),0)+1 from invoice_line_item),
194             now(),
195             $1,$2,$3,$4,$5,$6,$7,$8
196           )
197         `,
198         [invoice_id, li.product_id, li.quantity, li.unit_price, li.extended_price,
199          li.line_discount_percent, li.line_discount_amount, li.line_net_price],
200       );
201     }
202
203     await client.query("commit");
204     return { invoice_no: inv.rows[0].invoice_no };
205   } catch (err) {
206     await client.query("rollback");
207     throw err;
208   } finally {
209     client.release();
210   }
211 }
212
213 export async function deleteInvoice(idOrInvoiceNo) {
214   let id = idOrInvoiceNo;
215   if (typeof idOrInvoiceNo === "string" && String(idOrInvoiceNo).trim() !== "" &&
isNaN(Number(idOrInvoiceNo))) {
216     id = await resolveInvoiceId(String(idOrInvoiceNo).trim());
217     if (id == null) return null;
218   } else {
219     id = Number(idOrInvoiceNo);
220   }
221   await pool.query("delete from invoice where id=$1", [id]);
222   return { ok: true };
223 }
224
225 export async function updateInvoice(
226   idOrInvoiceNo,
227   { invoice_no, customer_code, sales_person_code, invoice_date, vat_rate, line_items },
228 ) {
229   let id = idOrInvoiceNo;
```

```
    if (typeof idOrInvoiceNo === "string" && String(idOrInvoiceNo).trim() !== "" &&
isNaN(Number(idOrInvoiceNo))) {
      id = await resolveInvoiceId(String(idOrInvoiceNo).trim());
      if (id == null) return null;
    } else {
      id = Number(idOrInvoiceNo);
    }

    const code = customer_code != null ? String(customer_code).trim() : "";
    const cust = await pool.query("SELECT id, credit_limit FROM customer WHERE code = $1", [code]);
    if (cust.rowCount === 0) throw new Error(`Customer not found: ${code}`);
    const customer_id = cust.rows[0].id;

    const client = await pool.connect();
    try {
      await client.query("begin");

      // Resolve sales_person_code !' id
      let sales_person_id = null;
      if (sales_person_code) {
        const sp = await client.query("SELECT id FROM sales_person WHERE code = $1", [sales_person_code]);
        if (sp.rowCount === 0) throw new Error(`Sales person not found: ${sales_person_code}`);
        sales_person_id = sp.rows[0].id;
      }

      const enriched = await enrichLineItems(client, line_items);

      const total_price = enriched.reduce((s, x) => s + x.extended_price, 0);
      const total_discount = enriched.reduce((s, x) => s + x.line_discount_amount, 0);
      const vat_amount = Math.round((total_price - total_discount) * Number(vat_rate) * 100) / 100;
      const amount_due = (total_price - total_discount) + vat_amount;

      if (cust.rows[0].credit_limit != null) {
        const limit = Number(cust.rows[0].credit_limit);
        if (amount_due > limit) {
          throw new Error(`Amount due (${amount_due}) exceeds customer credit limit (${limit}).`);
        }
      }

      let resolvedInvoiceNo = (invoice_no != null && String(invoice_no).trim() !== "") ?
String(invoice_no).trim() : null;
      if (resolvedInvoiceNo === null) {
        const cur = await client.query("SELECT invoice_no FROM invoice WHERE id=$1", [id]);
        if (cur.rowCount > 0) resolvedInvoiceNo = cur.rows[0].invoice_no;
        else resolvedInvoiceNo = `INV-${id}`;
      }

      await client.query(
        `
          UPDATE invoice
          SET invoice_no=$1, invoice_date=$2, customer_id=$3, sales_person_id=$4,
            total_amount=$5, vat=$6, amount_due=$7,
            vat_rate=$8, total_price=$9, total_discount=$10, vat_amount=$11
          WHERE id=$12
        `,
        [resolvedInvoiceNo, invoice_date, customer_id, sales_person_id,
         total_price, vat_amount, amount_due, Number(vat_rate), total_price, total_discount, vat_amount, id],
      );

      const keptLineIds = line_items.filter((li) => li.id != null && Number(li.id) > 0).map((li) =>
Number(li.id));

      if (keptLineIds.length > 0) {
        await client.query(
          "DELETE FROM invoice_line_item WHERE invoice_id = $1 AND id != ALL($2::bigint[])",
          [id, keptLineIds],
        );
      } else {
        await client.query("DELETE FROM invoice_line_item WHERE invoice_id = $1", [id]);
      }

      for (const li of enriched) {
        const lineId = li.id != null && Number(li.id) > 0 ? Number(li.id) : null;
        if (lineId) {
          await client.query(
            `
              UPDATE invoice_line_item
              SET product_id=$1, quantity=$2, unit_price=$3, extended_price=$4,
                line_discount_percent=$5, line_discount_amount=$6, line_net_price=$7
```

```
              WHERE id=$8 AND invoice_id=$9
            `,
            [li.product_id, li.quantity, li.unit_price, li.extended_price,
             li.line_discount_percent, li.line_discount_amount, li.line_net_price, lineId, id],
          );
        } else {
          await client.query(
            `
              INSERT INTO invoice_line_item (id, created_at, invoice_id, product_id, quantity, unit_price,
extended_price,
                line_discount_percent, line_discount_amount, line_net_price)
              VALUES (
                (select coalesce(max(id),0)+1 from invoice_line_item),
                now(),
                $1,$2,$3,$4,$5,$6,$7,$8
              )
            `,
            [id, li.product_id, li.quantity, li.unit_price, li.extended_price,
             li.line_discount_percent, li.line_discount_amount, li.line_net_price],
          );
        }
      }

      await client.query("commit");
      const inv = await pool.query("SELECT invoice_no FROM invoice WHERE id = $1", [id]);
      return { invoice_no: inv.rows[0]?.invoice_no };
    } catch (err) {
      await client.query("rollback");
      throw err;
    } finally {
      client.release();
    }
  }

```

Client Configuration API

```
import { http } from "./http.js";

export async function getConfig(key) {
  const res = await http(`/api/config/${encodeURIComponent(key)}`);
  return res.data.value;
}

```

Client Sales Person API (CRUD)

```
import { http } from "./http.js";

export async function listSalesPersons(params = {}) {
  const query = new URLSearchParams(params).toString();
  const res = await http(`/api/sales-persons${query ? `?${query}` : ""}`);
  return { data: res.data, ...(res.meta || {}) };
}

export async function getSalesPerson(code) {
  const res = await http(`/api/sales-persons/${encodeURIComponent(code)}`);
  return res.data;
}

export async function createSalesPerson(body) {
  const res = await http("/api/sales-persons", { method: "POST", body: JSON.stringify(body) });
  return res.data;
}

export async function updateSalesPerson(code, body) {
  const res = await http(`/api/sales-persons/${encodeURIComponent(code)}`, { method: "PUT", body:
JSON.stringify(body) });
  return res.data;
}

export async function deleteSalesPerson(code) {
  const res = await http(`/api/sales-persons/${encodeURIComponent(code)}`, { method: "DELETE" });
  return res.data;
}

```

Sales Person List Page

```jsx
import React from "react";
import { toast } from "react-toastify";
import { listSalesPersons, deleteSalesPerson } from "../../api/salesPersons.api.js";
import DataList from "../../components/DataList.jsx";
import { ConfirmModal } from "../../components/Modal.jsx";

export default function SalesPersonList() {
    const fetchData = React.useCallback((params) => listSalesPersons(params), []);
    const [confirmModal, setConfirmModal] = React.useState({ isOpen: false, id: null });
    const [refreshTrigger, setRefreshTrigger] = React.useState(0);

    const handleDelete = (id) => {
        setConfirmModal({ isOpen: true, id });
    };

    const confirmDelete = async () => {
        try {
            await deleteSalesPerson(confirmModal.id);
            setConfirmModal({ isOpen: false, id: null });
            setRefreshTrigger((t) => t + 1);
            toast.success("Sales person deleted.");
        } catch (e) {
            toast.error(String(e.message || e));
            setConfirmModal({ isOpen: false, id: null });
        }
    };

    const columns = [
        { key: "code", label: "Code" },
        { key: "name", label: "Name" },
        { key: "start_work_date", label: "Start Date", render: (v) => v ? String(v).slice(0, 10) : "-" },
    ];

    return (
        <>
            <ConfirmModal
                isOpen={confirmModal.isOpen}
                onClose={() => setConfirmModal({ isOpen: false, id: null })}
                onConfirm={confirmDelete}
                closeOnConfirm={false}
                title="Delete Sales Person"
                message="Are you sure you want to delete this sales person?"
                confirmText="Delete"
            />
            <DataList
                title="Sales Persons"
                fetchData={fetchData}
                columns={columns}
                searchPlaceholder="Search code, name..."
                itemName="sales persons"
                basePath="/sales-persons"
                itemKey="code"
                onDelete={handleDelete}
                refreshTrigger={refreshTrigger}
            />
        </>
    );
}

```

Sales Person Create/Edit/View Page

```jsx
import React from "react";
import { useNavigate, Link, useParams } from "react-router-dom";
import { toast } from "react-toastify";
import { getSalesPerson, createSalesPerson, updateSalesPerson } from "../../api/salesPersons.api.js";
import Loading from "../../components/Loading.jsx";

export default function SalesPersonPage({ mode: propMode }) {
    const { id } = useParams();
    const mode = propMode || (id ? "view" : "create");
    const nav = useNavigate();

    const [code, setCode] = React.useState("");
    const [name, setName] = React.useState("");
    const [startWorkDate, setStartWorkDate] = React.useState("");
    const [autoCode, setAutoCode] = React.useState(true);
    const [err, setErr] = React.useState("");
    const [submitting, setSubmitting] = React.useState(false);
    const [loading, setLoading] = React.useState(mode !== "create");

    React.useEffect(() => {
        if (mode !== "create") {
            getSalesPerson(id)
                .then((sp) => {
                    if (!sp) { setErr("Sales person not found"); setLoading(false); return; }
                    setCode(sp.code || "");
                    setName(sp.name || "");
                    setStartWorkDate(sp.start_work_date ? String(sp.start_work_date).slice(0, 10) : "");
                    setLoading(false);
                })
                .catch((e) => {
                    setErr(String(e.message || e));
                    setLoading(false);
                });
        }
    }, [id, mode]);

    async function handleSubmit(e) {
        e.preventDefault();
        setErr("");
        setSubmitting(true);
        try {
            const payload = { name, start_work_date: startWorkDate || null };
            if (mode === "create") {
                if (!autoCode) payload.code = code;
                await createSalesPerson(payload);
                toast.success("Sales person created.");
            } else {
                await updateSalesPerson(id, payload);
                toast.success("Sales person updated.");
            }
            nav("/sales-persons");
        } catch (e) {
            const msg = String(e.message || e);
            setErr(msg);
            toast.error(msg);
        } finally {
            setSubmitting(false);
        }
    }

    if (loading) return <Loading size="large" />;

    const isView = mode === "view";
    const isCreate = mode === "create";
    const titles = { create: "Create Sales Person", view: "Sales Person Details", edit: "Edit Sales Person" };
    const title = titles[mode];

    if (isView) {
        return (
            <div>
                <div className="page-header">
                    <h3 className="page-title">{title}</h3>
                    <div className="flex gap-4">
                        <Link to="/sales-persons" className="btn btn-outline">!• Back</Link>
                        <Link to={`/sales-persons/${id}/edit`} className="btn btn-primary">Edit</Link>
                    </div>
```

```
                </div>
                <div className="card">
                    <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: "2rem" }}>
                        <div>
                            <h4 style={{ marginBottom: "1.5rem", color: "var(--primary)" }}>Basic
Information</h4>
                            <div style={{ marginBottom: "1rem" }}>
                                <div style={{ fontSize: "0.8rem", color: "var(--text-muted)",
marginBottom: "4px" }}>Code</div>
                                <div style={{ fontWeight: 600, fontSize: "1.1rem" }}>{code}</div>
                            </div>
                            <div style={{ marginBottom: "1rem" }}>
                                <div style={{ fontSize: "0.8rem", color: "var(--text-muted)",
marginBottom: "4px" }}>Name</div>
                                <div style={{ fontWeight: 600, fontSize: "1.1rem" }}>{name}</div>
                            </div>
                            <div style={{ marginBottom: "1rem" }}>
                                <div style={{ fontSize: "0.8rem", color: "var(--text-muted)",
marginBottom: "4px" }}>Start Date</div>
                                <div>{startWorkDate || "-"}</div>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        );
    }

    return (
        <div>
            <div className="page-header">
                <h3 className="page-title">{title}</h3>
                <Link to="/sales-persons" className="btn btn-outline">
                    <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><line x1="19" y1="12" x2="5"
y2="12"></line><polyline points="12 19 5 12 12 5"></polyline></svg>
                    Back
                </Link>
            </div>
            {err && <div className="alert alert-error">{err}</div>}
            <div className="card">
                <form onSubmit={handleSubmit}>
                    <div style={{ display: "grid", gridTemplateColumns: "1fr 2fr", gap: "1rem",
marginBottom: "1rem" }}>
                        <div className="form-group">
                            <label className="form-label">{isCreate && autoCode ? "Code" : <>Code <span
className="required-marker">*</span></>}</label>
                            {isCreate ? (
                                <div className="flex gap-2">
                                    <input
                                        className="form-control"
                                        disabled={autoCode}
                                        value={code}
                                        onChange={(e) => setCode(e.target.value)}
                                        placeholder="SP001"
                                    />
                                    <div className="form-inline-option">
                                        <input type="checkbox" checked={autoCode} onChange={(e) =>
setAutoCode(e.target.checked)} id="sp_auto" />
                                        <label htmlFor="sp_auto">Auto</label>
                                    </div>
                                </div>
                            ) : (
                                <input className="form-control" value={code} disabled readOnly
placeholder="SP001" />
                            )}
                        </div>
                        <div className="form-group">
                            <label className="form-label">Name <span className="required-marker">*</span></
label>
                            <input className="form-control" required value={name} onChange={(e) =>
setName(e.target.value)} placeholder="Sales Person Name" />
                        </div>
                    </div>
                    <div style={{ display: "grid", gridTemplateColumns: "1fr 2fr", gap: "1rem",
marginBottom: "1.5rem" }}>
                        <div className="form-group">
                            <label className="form-label">Start Date</label>
                            <input type="date" className="form-control" value={startWorkDate}
onChange={(e) => setStartWorkDate(e.target.value)} />
```

```
                        </div>
                    </div>
                    <button type="submit" className="btn btn-primary" disabled={submitting}>
                        {submitting ? (isCreate ? "Creating..." : "Saving...") : (isCreate ? "Create Sales Person" : "Save Changes")}
                    </button>
                </form>
            </div>
        </div>
    );
}

```

App Router & Nav (Sales Persons)

```
// App entry: defines which URL path renders which page (routes)
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter, Routes, Route, NavLink, Navigate } from "react-router-dom";
import { ToastContainer } from "react-toastify";
import "react-toastify/dist/ReactToastify.css";
import InvoiceList from "./pages/invoices/InvoiceList.jsx";
import InvoicePage from "./pages/invoices/InvoicePage.jsx";
import CustomerList from "./pages/customers/CustomerList.jsx";
import CustomerPage from "./pages/customers/CustomerPage.jsx";
import ProductList from "./pages/products/ProductList.jsx";
import ProductPage from "./pages/products/ProductPage.jsx";
import Reports from "./pages/reports/Reports.jsx";
import SalesPersonList from "./pages/salesPersons/SalesPersonList.jsx";
import SalesPersonPage from "./pages/salesPersons/SalesPersonPage.jsx";
import { http } from "./api/http.js";
import "./index.css";

// Collapsible submenu (e.g. Reports !’ Product Sales, Monthly Sales)
function SubMenu({ icon, label, children, basePath }) {
  const location = window.location.pathname;
  const isActive = location.startsWith(basePath);
  const [isOpen, setIsOpen] = React.useState(isActive);

  return (
    <div className={`nav-group ${isOpen ? 'open' : ''}`}>
      <button
        type="button"
        className={`nav-item nav-group-toggle ${isActive ? 'active' : ''}`}
        onClick={() => setIsOpen(!isOpen)}
      >
        {icon}
        <span style={{ flex: 1 }}>{label}</span>
        <svg
          width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
          strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"
          style={{ transition: 'transform 0.2s', transform: isOpen ? 'rotate(180deg)' : 'rotate(0)' }}
        >
          <polyline points="6 9 12 15 18 9"></polyline>
        </svg>
      </button>
      <div className="nav-submenu" style={{ display: isOpen ? 'block' : 'none' }}>
        {children}
      </div>
    </div>
  );
}

function Sidebar() {
  // NavLink injects isActive when the current URL matches this link
  const getLinkClass = ({ isActive }) => isActive ? "nav-item active" : "nav-item";
  const getSubLinkClass = ({ isActive }) => isActive ? "nav-subitem active" : "nav-subitem";

  return (
    <aside className="sidebar no-print">
      <div className="sidebar-header">
        <div className="brand">
          <svg width="24" height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
            <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 0 2 2h12a2 2 0 0 0 2-2V8z"></path>
            <polyline points="14 2 14 8 20 8"></polyline>
            <line x1="16" y1="13" x2="8" y2="13"></line>
            <line x1="16" y1="17" x2="8" y2="17"></line>
            <polyline points="10 9 9 9 8 9"></polyline>
          </svg>
          InvoiceDoc v2
        </div>
      </div>
      <nav className="sidebar-nav">
        <NavLink to="/invoices" className={getLinkClass}>
          <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><rect x="3" y="4" width="18"
height="18" rx="2" ry="2"></rect><line x1="16" y1="2" x2="16" y2="6"></line><line x1="8" y1="2" x2="8" y2="6"></
line><line x1="3" y1="10" x2="21" y2="10"></line></svg>
          Invoices
        </NavLink>
        <NavLink to="/customers" className={getLinkClass}>
```

```
            <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"></path><circle cx="9" cy="7" r="4"></circle><path d="M23 21v-2a4 4 0 0 0-3-3.87"></path><path d="M16 3.13a4 4 0 0 1 0 7.75"></path></svg>
            Customers
          </NavLink>
          <NavLink to="/products" className={getLinkClass}>
            <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M21 16V8a2 2 0 0 0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4A2 2 0 0 0 3 8v8a2 2 0 0 0 1 1.73l7 4a2 2 0 0 0 2 0l7-4A2 2 0 0 0 21 16z"></path><polyline points="3.27 6.96 12 12.01 20.73 6.96"></polyline><line x1="12" y1="22.08" x2="12" y2="12"></line></svg>
            Products
          </NavLink>
          <NavLink to="/sales-persons" className={getLinkClass}>
            <svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><circle cx="12" cy="8" r="4"></circle><path d="M4 20c0-4 3.6-7 8-7s8 3 8 7"></path></svg>
            Sales Persons
          </NavLink>

          <SubMenu
            basePath="/reports"
            icon={<svg style={{ marginRight: 10 }} width="18" height="18" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><line x1="18" y1="20" x2="18" y2="10"></line><line x1="12" y1="20" x2="12" y2="4"></line><line x1="6" y1="20" x2="6" y2="14"></line></svg>}
            label="Reports"
          >
            <NavLink to="/reports/product-sales" className={getSubLinkClass}>
              Product Sales
            </NavLink>
            <NavLink to="/reports/monthly-sales" className={getSubLinkClass}>
              Monthly Sales
            </NavLink>
            <NavLink to="/reports/customer-sales" className={getSubLinkClass}>
              Customer Buying
            </NavLink>
          </SubMenu>
        </nav>
      </aside>
    );
  }

  // Banner when repo has newer commits (server returns updateAvailable if GIT_SHA is set and behind GitHub)
  function UpdateBanner() {
    const [show, setShow] = React.useState(false);
    const [repoUrl, setRepoUrl] = React.useState("");
    const DISMISS_KEY = "invoicedoc2-update-banner-dismissed";

    React.useEffect(() => {
      if (localStorage.getItem(DISMISS_KEY)) return;
      http("/api/updates-check")
        .then((data) => {
          if (data.updateAvailable && data.repoUrl) {
            setShow(true);
            setRepoUrl(data.repoUrl);
          }
        })
        .catch(() => {});
    }, []);

    const dismiss = () => {
      localStorage.setItem(DISMISS_KEY, "1");
      setShow(false);
    };

    if (!show) return null;
    return (
      <div className="update-banner no-print">
        <span>
          Update available: the repo has new changes — run <code>git pull origin</code> in your project folder to get the latest, or check{" "}
          <a href={repoUrl} target="_blank" rel="noreferrer">
            GitHub
          </a>
        </span>
        <button type="button" className="update-banner-dismiss" onClick={dismiss} aria-label="Close">
          ×
```

```
        </button>
      </div>
    );
  }

  // Mobile warning overlay
  function MobileWarning() {
    const [dismissed, setDismissed] = React.useState(false);
    if (dismissed) return null;

    return (
      <div className="mobile-warning">
        <div className="mobile-warning-content">
          <svg width="48" height="48" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" strokeLinejoin="round">
            <rect x="5" y="2" width="14" height="20" rx="2" ry="2"></rect>
            <line x1="12" y1="18" x2="12.01" y2="18"></line>
          </svg>
          <h3>Whoa there! Ø=Üñ</h3>
          <p>TA was too busy touching grass to make this mobile-friendly.<br/>Please grab a laptop. We
believe in you.</p>
          <button className="btn btn-primary" onClick={() => setDismissed(true)}>
            Okay, I'll suffer
          </button>
        </div>
      </div>
    );
  }

  function Layout({ children }) {
    return (
      <div className="layout-container">
        <ToastContainer position="bottom-right" autoClose={4000} hideProgressBar={false} theme="light" />
        <MobileWarning />
        <UpdateBanner />
        <div className="top-banner no-print">
          This is a sample term project for CPE241 Database Systems. Some functions may be incomplete. Click
          <a href="https://google.com" target="_blank" rel="noreferrer">here for questions</a>
        </div>
        <div className="app-layout">
          <Sidebar />
          <main className="main-wrapper">
            {children}
          </main>
        </div>
      </div>
    );
  }

  // Route definitions: path !' component (e.g. /invoices !' InvoiceList)
  ReactDOM.createRoot(document.getElementById("root")).render(
    <React.StrictMode>
      <BrowserRouter>
        <Routes>
          {/* Default route !' invoices list */}
          <Route path="/" element={<Navigate to="/invoices" replace />} />
          <Route path="/invoices" element={<Layout><InvoiceList /></Layout>} />
          <Route path="/invoices/new" element={<Layout><InvoicePage mode="create" /></Layout>} />
          <Route path="/invoices/:id" element={<Layout><InvoicePage mode="view" /></Layout>} />
          <Route path="/invoices/:id/edit" element={<Layout><InvoicePage mode="edit" /></Layout>} />
          <Route path="/customers" element={<Layout><CustomerList /></Layout>} />
          <Route path="/customers/new" element={<Layout><CustomerPage mode="create" /></Layout>} />
          <Route path="/customers/:id" element={<Layout><CustomerPage mode="view" /></Layout>} />
          <Route path="/customers/:id/edit" element={<Layout><CustomerPage mode="edit" /></Layout>} />
          <Route path="/products" element={<Layout><ProductList /></Layout>} />
          <Route path="/products/new" element={<Layout><ProductPage mode="create" /></Layout>} />
          <Route path="/products/:id" element={<Layout><ProductPage mode="view" /></Layout>} />
          <Route path="/products/:id/edit" element={<Layout><ProductPage mode="edit" /></Layout>} />
          <Route path="/sales-persons" element={<Layout><SalesPersonList /></Layout>} />
          <Route path="/sales-persons/new" element={<Layout><SalesPersonPage mode="create" /></Layout>} />
          <Route path="/sales-persons/:id" element={<Layout><SalesPersonPage mode="view" /></Layout>} />
          <Route path="/sales-persons/:id/edit" element={<Layout><SalesPersonPage mode="edit" /></Layout>} />
          <Route path="/reports" element={<Navigate to="/reports/product-sales" replace />} />
          <Route path="/reports/product-sales" element={<Layout><Reports type="product-sales" /></Layout>} />
          <Route path="/reports/monthly-sales" element={<Layout><Reports type="monthly-sales" /></Layout>} />
          <Route path="/reports/customer-sales" element={<Layout><Reports type="customer-sales" /></
Layout>} />
        </Routes>
      </BrowserRouter>
```

```
216    </React.StrictMode>
217  );
218
```

Line Items Editor (discount columns)

```
// Line items table: add/remove/reorder rows, insert row between. Drag and drop and up/down buttons.
// Each row is a separate component (LineItemRow.jsx) for easier reading.
import React, { useState } from "react";
import { toast } from "react-toastify";
import { formatBaht } from "../utils.js";
import { getProduct } from "../api/products.api.js";
import ProductPickerModal from "./ProductPickerModal.jsx";
import LineItemRow from "./LineItemRow.jsx";

export default function LineItemsEditor({ value, onChange }) {
    const items = value;
    const [dragIndex, setDragIndex] = useState(null);
    const [dragOverIndex, setDragOverIndex] = useState(null);
    const [productPickerOpen, setProductPickerOpen] = useState(false);
    const [pickerRowIndex, setPickerRowIndex] = useState(0);
    const [productErrorRow, setProductErrorRow] = useState(null); // row index that has "product not found"

    // Update one row by index
    function update(i, patch) {
        const next = items.map((x, idx) => (idx === i ? { ...x, ...patch } : x));
        onChange(next);
    }

    // Add a new row with empty product (user must select) at the end
    function addRow() {
        onChange([
            ...items,
            { product_code: "", product_name: "", quantity: 1, unit_price: 0, line_discount_percent: 0 },
        ]);
    }

    // Insert a new empty row after index i (add row in between)
    function insertRowAfter(i) {
        const newRow = { product_code: "", product_name: "", quantity: 1, unit_price: 0,
line_discount_percent: 0 };
        const next = [...items.slice(0, i + 1), newRow, ...items.slice(i + 1)];
        onChange(next);
    }

    // Remove a row by index
    function removeRow(i) {
        onChange(items.filter((_, idx) => idx !== i));
    }

    // Move item up
    function moveUp(i) {
        if (i <= 0) return;
        const newItems = [...items];
        [newItems[i - 1], newItems[i]] = [newItems[i], newItems[i - 1]];
        onChange(newItems);
    }

    // Move item down
    function moveDown(i) {
        if (i >= items.length - 1) return;
        const newItems = [...items];
        [newItems[i], newItems[i + 1]] = [newItems[i + 1], newItems[i]];
        onChange(newItems);
    }

    // Drag and drop handlers
    function handleDragStart(e, index) {
        setDragIndex(index);
        e.dataTransfer.effectAllowed = 'move';
        e.dataTransfer.setData('text/plain', index);
    }

    function handleDragOver(e, index) {
        e.preventDefault();
        e.dataTransfer.dropEffect = 'move';
        setDragOverIndex(index);
    }

    function handleDragLeave() {
        setDragOverIndex(null);
    }

```

```
    function handleDrop(e, dropIndex) {
        e.preventDefault();
        if (dragIndex === null || dragIndex === dropIndex) {
            setDragIndex(null);
            setDragOverIndex(null);
            return;
        }

        const newItems = [...items];
        const [removed] = newItems.splice(dragIndex, 1);
        newItems.splice(dropIndex, 0, removed);
        onChange(newItems);

        setDragIndex(null);
        setDragOverIndex(null);
    }

    function handleDragEnd() {
        setDragIndex(null);
        setDragOverIndex(null);
    }

    function onPickProduct(i, productCode, productData) {
        setProductErrorRow(null);
        if (!productData) {
            update(i, { product_code: "", product_name: "", product_label: "", units_code: "", unit_price:
0 });
            return;
        }

        const existingIndex = items.findIndex((it, idx) => idx !== i && String(it.product_code) ===
String(productData.code));
        if (existingIndex !== -1) {
            const currentQty = Number(items[i].quantity || 1);
            const existingQty = Number(items[existingIndex].quantity || 0);
            const newItems = items.filter((_, idx) => idx !== i);
            newItems[existingIndex > i ? existingIndex - 1 : existingIndex] = {
                ...newItems[existingIndex > i ? existingIndex - 1 : existingIndex],
                quantity: existingQty + currentQty
            };
            onChange(newItems);
        } else {
            update(i, {
                product_code: productData.code,
                product_name: productData.name ?? "",
                product_label: productData.label,
                units_code: productData.units_code,
                unit_price: Number(productData.unit_price || 0)
            });
        }
    }

    function handleProductCodeBlur(i) {
        const code = String(items[i]?.product_code ?? "").trim();
        setProductErrorRow(null);
        if (!code) return;
        getProduct(code)
            .then((p) => {
                update(i, {
                    product_code: p.code,
                    product_name: p.name ?? "",
                    product_label: `${p.code} - ${p.name}`,
                    units_code: p.units_code ?? "",
                    unit_price: Number(p.unit_price ?? 0)
                });
            })
            .catch(() => {
                update(i, { product_name: "", product_label: "", units_code: "", unit_price: 0 });
                setProductErrorRow(i);
                toast.error(`Product not found: ${code}`);
            });
    }

    function computeExtended(it) {
        const q = Number(it.quantity || 0);
        const up = Number(it.unit_price || 0);
        return q * up;
    }

```

```
    function computeDiscountAmount(it) {
        const extended = computeExtended(it);
        const pct = Number(it.line_discount_percent || 0);
        return Math.round(extended * pct / 100 * 100) / 100;
    }

    function computeNetPrice(it) {
        return Math.round((computeExtended(it) - computeDiscountAmount(it)) * 100) / 100;
    }

    const totalPrice = items.reduce((s, it) => s + computeExtended(it), 0);
    const totalDiscount = items.reduce((s, it) => s + computeDiscountAmount(it), 0);

    return (
        <div className="card">
            <div style={{
                display: "flex",
                justifyContent: "space-between",
                alignItems: "center",
                marginBottom: 12,
                paddingBottom: 12,
                borderBottom: '1px solid var(--border)'
            }}>
                <div>
                    <h4 style={{ margin: 0, fontSize: '0.95rem', fontWeight: 600, color: 'var(--text-
main)' }}>Line Items</h4>
                    <span style={{ fontSize: '0.8rem', color: 'var(--text-muted)' }}>
                        {items.length} item{items.length !== 1 ? 's' : ''} added • Drag to reorder
                    </span>
                </div>
                <button
                    type="button"
                    onClick={addRow}
                    className="btn btn-primary"
                    style={{ padding: '8px 16px' }}
                >
                    <svg style={{ marginRight: 6 }} width="14" height="14" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2.5" strokeLinecap="round" strokeLinejoin="round">
                        <line x1="12" y1="5" x2="12" y2="19"></line>
                        <line x1="5" y1="12" x2="19" y2="12"></line>
                    </svg>
                    Add Item
                </button>
            </div>

            <div className="table-container">
                <table className="modern-table" style={{ fontSize: '0.9rem' }}>
                    <thead>
                        <tr>
                            <th style={{ width: '60px' }} className="text-center">#</th>
                            <th style={{ width: '14%' }}>Product Code <span className="required-marker">*</
span></th>
                            <th style={{ width: '18%' }}>Product Name</th>
                            <th style={{ width: '6%' }} className="text-center">Unit</th>
                            <th style={{ width: '8%' }} className="text-right">Qty <span
className="required-marker">*</span></th>
                            <th style={{ width: '10%' }} className="text-right">Unit Price <span
className="required-marker">*</span></th>
                            <th style={{ width: '10%' }} className="text-right">Extended</th>
                            <th style={{ width: '8%' }} className="text-right">Disc %</th>
                            <th style={{ width: '10%' }} className="text-right">Disc Amt</th>
                            <th style={{ width: '10%' }} className="text-right">Net Price</th>
                            <th style={{ width: '100px' }} className="text-center">Actions</th>
                        </tr>
                    </thead>
                    <tbody>
                        {items.map((it, i) => (
                            <LineItemRow
                                key={i}
                                index={i}
                                item={it}
                                itemsLength={items.length}
                                update={update}
                                removeRow={removeRow}
                                moveUp={moveUp}
                                moveDown={moveDown}
                                insertRowAfter={insertRowAfter}
                                isDragging={dragIndex === i}
                                isDragOver={dragOverIndex === i && dragIndex !== i}
```

```
                                    onDragStart={handleDragStart}
                                    onDragOver={handleDragOver}
                                    onDragLeave={handleDragLeave}
                                    onDrop={handleDrop}
                                    onDragEnd={handleDragEnd}
                                    productErrorRow={productErrorRow}
                                    setProductErrorRow={setProductErrorRow}
                                    onPickProduct={onPickProduct}
                                    onOpenPicker={(idx) => { setPickerRowIndex(idx);
setProductPickerOpen(true); }}
                                    onProductCodeBlur={handleProductCodeBlur}
                                    formatBaht={formatBaht}
                                    computeExtended={computeExtended}
                                    computeDiscountAmount={computeDiscountAmount}
                                    computeNetPrice={computeNetPrice}
                                />
                            ))}
                            {items.length === 0 && (
                                <tr>
                                    <td colSpan="11" style={{
                                        padding: 40,
                                        textAlign: 'center',
                                        color: 'var(--text-muted)'
                                    }}>
                                        <svg style={{ marginBottom: 12, opacity: 0.5 }} width="48" height="48"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.5" strokeLinecap="round"
strokeLinejoin="round">
                                            <path d="M21 16V8a2 2 0 0 0-1-1.73l-7-4a2 2 0 0 0-2 0l-7 4A2 2 0 0
0 3 8v8a2 2 0 0 0 1 1.73l7 4a2 2 0 0 0 2 0l7-4A2 2 0 0 0 21 16z"></path>
                                        </svg>
                                        <div>No items added yet</div>
                                        <div style={{ fontSize: '0.85rem', marginTop: 4 }}>Click "Add Item" to
start</div>
                                    </td>
                                </tr>
                            )}
                        </tbody>
                    </table>
                </div>

                {/* Subtotal Footer */}
                {items.length > 0 && (
                    <div style={{
                        display: 'flex',
                        justifyContent: 'flex-end',
                        marginTop: 16,
                        paddingTop: 16,
                        borderTop: '2px solid var(--border)'
                    }}>
                        <div style={{
                            display: 'flex',
                            alignItems: 'center',
                            gap: 32,
                            padding: '12px 20px',
                            background: 'var(--bg-body)',
                            borderRadius: 'var(--radius-sm)'
                        }}>
                            <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'flex-end',
gap: 4 }}>
                                <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>
                                    Total Price ({items.length} item{items.length !== 1 ? 's' : ''})
                                </span>
                                <span style={{ fontSize: '1.1rem', fontWeight: 700, color: 'var(--primary)' }}>
                                    {formatBaht(totalPrice)}
                                </span>
                            </div>
                            {totalDiscount > 0 && (
                                <div style={{ display: 'flex', flexDirection: 'column', alignItems: 'flex-
end', gap: 4 }}>
                                    <span style={{ fontSize: '0.85rem', color: 'var(--text-muted)' }}>Total
Discount</span>
                                    <span style={{ fontSize: '1.1rem', fontWeight: 700, color: '#ef4444' }}>
                                        -{formatBaht(totalDiscount)}
                                    </span>
                                </div>
                            )}
                        </div>
                    </div>
                )}
```

```
299
300             <ProductPickerModal
301                 isOpen={productPickerOpen}
302                 onClose={() => setProductPickerOpen(false)}
303                 initialSearch={items[pickerRowIndex]?.product_code ?? ""}
304                 onSelect={(row) => {
305                     const productData = {
306                         code: row.code,
307                         name: row.name,
308                         label: `${row.code} - ${row.name}`,
309                         units_code: row.units_code,
310                         unit_price: Number(row.unit_price || 0),
311                     };
312                     onPickProduct(pickerRowIndex, row.code, productData);
313                 }}
314             />
315         </div>
316     );
317 }
318
```

Line Item Row (Disc %, Disc Amt, Net Price)

```
// Single row in the line items table: product code, name, qty, unit price, discount, reorder/remove buttons
import React from "react";
import { formatBaht } from "../utils.js";

const actionBtnStyle = {
  background: "none",
  border: "none",
  cursor: "pointer",
  padding: 4,
  borderRadius: 4,
  color: "#666",
  transition: "all 0.15s",
  display: "flex",
  alignItems: "center",
  justifyContent: "center",
};
const disabledBtnStyle = { ...actionBtnStyle, cursor: "not-allowed", color: "#ddd" };

const DragIcon = () => (
  <svg width="16" height="16" viewBox="0 0 24 24" fill="currentColor">
    <circle cx="9" cy="6" r="1.5" />
    <circle cx="15" cy="6" r="1.5" />
    <circle cx="9" cy="12" r="1.5" />
    <circle cx="15" cy="12" r="1.5" />
    <circle cx="9" cy="18" r="1.5" />
    <circle cx="15" cy="18" r="1.5" />
  </svg>
);
const ArrowUpIcon = () => (
  <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5" strokeLinecap="round" strokeLinejoin="round">
    <polyline points="18 15 12 9 6 15"></polyline>
  </svg>
);
const ArrowDownIcon = () => (
  <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2.5" strokeLinecap="round" strokeLinejoin="round">
    <polyline points="6 9 12 15 18 9"></polyline>
  </svg>
);

export default function LineItemRow({
  index,
  item,
  itemsLength,
  update,
  removeRow,
  moveUp,
  moveDown,
  insertRowAfter,
  isDragging,
  isDragOver,
  onDragStart,
  onDragOver,
  onDragLeave,
  onDrop,
  onDragEnd,
  productErrorRow,
  setProductErrorRow,
  onPickProduct,
  onOpenPicker,
  onProductCodeBlur,
  formatBaht,
  computeExtended,
  computeDiscountAmount,
  computeNetPrice,
}) {
  const i = index;
  const it = item;
  return (
    <tr
      draggable
      onDragStart={(e) => onDragStart(e, i)}
      onDragOver={(e) => onDragOver(e, i)}
      onDragLeave={onDragLeave}
      onDrop={(e) => onDrop(e, i)}
```

```
        onDragEnd={onDragEnd}
        style={{
          opacity: isDragging ? 0.5 : 1,
          background: isDragOver ? "var(--primary-light, #e0f2fe)" : "transparent",
          transition: "background 0.15s, opacity 0.15s",
          cursor: "grab",
        }}
      >
        <td className="text-center">
          <div style={{ display: "flex", alignItems: "center", justifyContent: "center", gap: 4, color:
"var(--text-muted)" }}>
            <span style={{ cursor: "grab", color: "#aaa" }} title="Drag to reorder">
              <DragIcon />
            </span>
            <span style={{ fontWeight: 600, fontSize: "0.85rem", color: "var(--text-muted)", minWidth: 20 }}
>{i + 1}</span>
          </div>
        </td>
        <td>
          <div>
            <div style={{ display: "flex", gap: 6, alignItems: "stretch" }}>
              <input
                type="text"
                className="form-control"
                value={it.product_code || ""}
                onChange={(e) => {
                  update(i, { product_code: e.target.value });
                  if (productErrorRow === i) setProductErrorRow(null);
                }}
                onBlur={() => onProductCodeBlur(i)}
                placeholder="e.g. P001"
                style={{ flex: 1, minWidth: 0, padding: "6px 10px", fontSize: "0.9rem", borderColor:
productErrorRow === i ? "#ef4444" : undefined }}
              />
              <button
                type="button"
                className="btn btn-primary"
                style={{ flexShrink: 0, padding: "6px 12px", fontSize: "0.8rem" }}
                onClick={() => onOpenPicker(i)}
                title="List of Values"
              >
                LoV
              </button>
              {it.product_code && (
                <button
                  type="button"
                  onClick={(e) => {
                    e.stopPropagation();
                    onPickProduct(i, "", null);
                    setProductErrorRow(null);
                  }}
                  title="Clear"
                  style={{
                    padding: "0 8px",
                    border: "1px solid var(--border)",
                    borderRadius: "var(--radius-sm)",
                    background: "var(--bg-body)",
                    color: "var(--text-muted)",
                    cursor: "pointer",
                    fontSize: "1.1rem",
                    lineHeight: 1,
                  }}
                >
                  ×
                </button>
              )}
            </div>
            {productErrorRow === i && (
              <span style={{ fontSize: "0.75rem", color: "#ef4444", marginTop: 4, display: "block" }}
>Product not found</span>
            )}
          </div>
        </td>
        <td>
          <input
            type="text"
            className="form-control"
            disabled
            readOnly
```

```
            value={it.product_name ?? it.product_label?.replace(/^[^\-]+\s*-\s*/, "") ?? ""}
            placeholder="—"
            style={{ padding: "6px 10px", fontSize: "0.9rem", background: "var(--bg-body)" }}
          />
        </td>
        <td className="text-center">
          <span
            style={{
              display: "inline-block",
              padding: "4px 10px",
              background: "var(--bg-body)",
              borderRadius: 4,
              fontSize: "0.8rem",
              color: "var(--text-muted)",
              fontWeight: 500,
            }}
          >
            {it.units_code || "-"}
          </span>
        </td>
        <td>
          <input
            type="number"
            step="0.01"
            min="0.01"
            value={it.quantity}
            onChange={(e) => update(i, { quantity: e.target.value })}
            className="form-control"
            style={{ textAlign: "right", padding: "8px 12px", fontSize: "0.9rem" }}
          />
        </td>
        <td>
          <div
            style={{
              textAlign: "right",
              padding: "8px 12px",
              background: "var(--bg-body)",
              borderRadius: "var(--radius-sm)",
              color: "var(--text-muted)",
              fontSize: "0.9rem",
            }}
          >
            {formatBaht(it.unit_price)}
          </div>
        </td>
        <td>
          <div style={{ textAlign: "right", fontWeight: 600, color: "var(--primary)", fontSize: "0.95rem" }}>
            {formatBaht(computeExtended(it))}
          </div>
        </td>
        <td>
          <input
            type="number"
            step="0.01"
            min="0"
            max="100"
            value={it.line_discount_percent ?? 0}
            onChange={(e) => update(i, { line_discount_percent: e.target.value })}
            className="form-control"
            style={{ textAlign: "right", padding: "8px 12px", fontSize: "0.9rem" }}
          />
        </td>
        <td>
          <div style={{ textAlign: "right", color: "#ef4444", fontSize: "0.9rem" }}>
            {computeDiscountAmount(it) > 0 ? formatBaht(computeDiscountAmount(it)) : "-"}
          </div>
        </td>
        <td>
          <div style={{ textAlign: "right", fontWeight: 600, color: "var(--text-main)", fontSize:
"0.95rem" }}>
            {formatBaht(computeNetPrice(it))}
          </div>
        </td>
        <td>
          <div style={{ display: "flex", alignItems: "center", justifyContent: "center", gap: 2 }}>
            <button type="button" onClick={() => moveUp(i)} disabled={i === 0} style={i === 0 ?
disabledBtnStyle : actionBtnStyle} title="Move up">
              <ArrowUpIcon />
            </button>
```

```
            <button
              type="button"
              onClick={() => moveDown(i)}
              disabled={i === itemsLength - 1}
              style={i === itemsLength - 1 ? disabledBtnStyle : actionBtnStyle}
              title="Move down"
            >
              <ArrowDownIcon />
            </button>
            <button type="button" onClick={() => insertRowAfter(i)} style={actionBtnStyle} title="Insert row
below">
              <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="2.5" strokeLinecap="round" strokeLinejoin="round">
                <line x1="12" y1="5" x2="12" y2="19"></line>
                <line x1="5" y1="12" x2="19" y2="12"></line>
              </svg>
            </button>
            <button
              type="button"
              onClick={() => removeRow(i)}
              disabled={itemsLength <= 1}
              style={{
                ...actionBtnStyle,
                color: itemsLength <= 1 ? "#ddd" : "#ef4444",
                cursor: itemsLength <= 1 ? "not-allowed" : "pointer",
              }}
              title="Remove item"
            >
              <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
                <polyline points="3 6 5 6 21 6"></polyline>
                <path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2-2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2 2v2"></
path>
              </svg>
            </button>
          </div>
        </td>
      </tr>
    );
  }
```

Invoice Form (vatPercent, config API, summary)

```
// Invoice Form (create/edit) with customer select and line items
// Example usage: <InvoiceForm onSubmit={...} />
import React from "react";
import LineItemsEditor from "./LineItemsEditor.jsx";
import CustomerPickerModal from "./CustomerPickerModal.jsx";
import SalesPersonPickerModal from "./SalesPersonPickerModal.jsx";
import { AlertModal } from "./Modal.jsx";
import { getCustomer } from "../api/customers.api.js";
import { getConfig } from "../api/configuration.api.js";
import { formatBaht } from "../utils.js";

export default function InvoiceForm({ onSubmit, submitting, initialData }) {
  // Local state for header fields and line items
  const [invoiceNo, setInvoiceNo] = React.useState("");
  const [customerCode, setCustomerCode] = React.useState("");
  const [invoiceDate, setInvoiceDate] = React.useState(new Date().toISOString().slice(0, 10));
  const [vatPercent, setVatPercent] = React.useState(7);
  const [items, setItems] = React.useState([{ product_code: "", quantity: 1, unit_price: 0,
line_discount_percent: 0 }]);
  const [alertModal, setAlertModal] = React.useState({ isOpen: false, title: "Validation Error", message:
"" });
  const [customerModalOpen, setCustomerModalOpen] = React.useState(false);
  const [customerDetails, setCustomerDetails] = React.useState(null);
  const [customerLoadError, setCustomerLoadError] = React.useState("");
  const [salesPersonCode, setSalesPersonCode] = React.useState("");
  const [salesPersonName, setSalesPersonName] = React.useState("");
  const [salesPersonModalOpen, setSalesPersonModalOpen] = React.useState(false);

  // Fetch vat_percent from config on create mode
  React.useEffect(() => {
    if (!initialData) {
      getConfig("vat_percent")
        .then((val) => setVatPercent(Number(val)))
        .catch(() => {});
    }
  }, []);

  // When customer code is set (from LoV or initialData), fetch name and address
  React.useEffect(() => {
    const code = String(customerCode || "").trim();
    if (!code) {
      setCustomerDetails(null);
      setCustomerLoadError("");
      return;
    }
    setCustomerLoadError("");
    let cancelled = false;
    getCustomer(code)
      .then((data) => {
        if (!cancelled) setCustomerDetails(data);
      })
      .catch(() => {
        if (!cancelled) {
          setCustomerDetails(null);
          setCustomerLoadError("Customer not found");
        }
      });
    return () => { cancelled = true; };
  }, [customerCode]);

  // Load customer by code on blur (user typed code)
  const handleCustomerCodeBlur = () => {
    const code = String(customerCode || "").trim();
    if (!code) {
      setCustomerDetails(null);
      setCustomerLoadError("");
      return;
    }
    setCustomerLoadError("");
    getCustomer(code)
      .then((data) => setCustomerDetails(data))
      .catch(() => {
        setCustomerDetails(null);
        setCustomerLoadError("Customer not found");
      });
  };

```

```
  React.useEffect(() => {
    if (initialData) {
      setInvoiceNo(initialData.invoice_no);
      setCustomerCode(initialData.customer_code || "");
      setSalesPersonCode(initialData.sales_person_code || "");
      setSalesPersonName(initialData.sales_person_name || "");
      const d = initialData.invoice_date ? new Date(initialData.invoice_date).toISOString().slice(0, 10) :
"";
      setInvoiceDate(d);
      setVatPercent(Math.round(Number(initialData.vat_rate || 0.07) * 100));
      const mappedItems = (initialData.line_items || []).map(li => ({
        line_item_id: li.line_item_id,
        product_code: li.product_code || "",
        product_label: li.product_label || `${li.product_code || ""} - ${li.product_name || ""}`.replace(/
^ - /, ""),
        units_code: li.units_code || "",
        quantity: li.quantity,
        unit_price: Number(li.unit_price),
        line_discount_percent: Number(li.line_discount_percent || 0),
      }));
      setItems(mappedItems.length > 0 ? mappedItems : [{ product_code: "", quantity: 1, unit_price: 0,
line_discount_percent: 0 }]);
    }
  }, [initialData]);

  const vatRate = Number(vatPercent) / 100;
  const totalPrice = items.reduce((s, it) => s + Number(it.quantity || 0) * Number(it.unit_price || 0), 0);
  const totalDiscount = items.reduce((s, it) => {
    const extended = Number(it.quantity || 0) * Number(it.unit_price || 0);
    const pct = Number(it.line_discount_percent || 0);
    return s + Math.round(extended * pct / 100 * 100) / 100;
  }, 0);
  const netPrice = totalPrice - totalDiscount;
  const vatAmount = Math.round(netPrice * vatRate * 100) / 100;
  const amountDue = netPrice + vatAmount;

  const [autoCode, setAutoCode] = React.useState(true);

  const hasEmptyProduct = items.some(it => !it.product_code);
  const hasEmptyCustomer = !String(customerCode || "").trim() || !customerDetails;

  const customerAddressDisplay = customerDetails
    ? [customerDetails.address_line1, customerDetails.address_line2,
customerDetails.country_name].filter(Boolean).join(", ") || ""
    : "";

  // Collect all validation errors; return empty array if valid.
  function validate() {
    const errs = [];
    if (!invoiceDate || String(invoiceDate).trim() === "") errs.push("Date should not be null");
    if (!String(customerCode || "").trim()) errs.push("Customer Code should not be null");
    else if (!customerDetails) errs.push("Customer must be selected from list (enter code and blur, or use
LoV)");
    if (!initialData && !autoCode && !String(invoiceNo || "").trim()) errs.push("Invoice No should not be
null");
    items.forEach((it, i) => {
      const row = i + 1;
      if (!String(it.product_code || "").trim()) errs.push(`Row ${row}: Product is required`);
      else {
        const q = Number(it.quantity);
        if (Number.isNaN(q) || q <= 0) errs.push(`Row ${row}: Product quantity should be a positive
number`);
        const p = Number(it.unit_price);
        if (it.unit_price !== "" && it.unit_price != null && (Number.isNaN(p) || p < 0)) errs.push(`Row
${row}: Price should be a positive number`);
      }
    });
    return errs;
  }

  // Build payload and pass to parent
  function handleSubmit(e) {
    e.preventDefault();
    const errors = validate();
    if (errors.length > 0) {
      setAlertModal({
        isOpen: true,
        title: "Save Failed.",
        message: (
```

```
147            <ul style={{ margin: 0, paddingLeft: 20, color: "var(--text-main)" }}>
148              {errors.map((msg, i) => (
149                <li key={i} style={{ marginBottom: 4 }}>{msg}</li>
150              ))}
151            </ul>
152          ),
153        });
154        return;
155      }
156
157      const payload = {
158        invoice_no: initialData ? invoiceNo.trim() : (autoCode ? "" : invoiceNo.trim()),
159        customer_code: String(customerCode).trim(),
160        sales_person_code: String(salesPersonCode || "").trim() || undefined,
161        invoice_date: invoiceDate,
162        vat_rate: Number(vatPercent) / 100,
163        line_items: items.map((x) => {
164          const out = {
165            product_code: String(x.product_code || "").trim(),
166            quantity: Number(x.quantity),
167            unit_price: x.unit_price === "" || x.unit_price === null ? undefined : Number(x.unit_price),
168            line_discount_percent: Number(x.line_discount_percent || 0),
169          };
170          if (x.line_item_id != null && Number(x.line_item_id) > 0) out.id = Number(x.line_item_id);
171          return out;
172        }),
173      };
174      onSubmit(payload);
175    }
176
177    return (
178      <>
179        <AlertModal
180          isOpen={alertModal.isOpen}
181          onClose={() => setAlertModal((prev) => ({ ...prev, isOpen: false }))}
182          title={alertModal.title}
183          message={alertModal.message}
184        />
185        <form onSubmit={handleSubmit} className="invoice-form">
186        <div className="invoice-form-grid" style={{ display: "grid", gridTemplateColumns: "1fr 280px", gap:
16, marginBottom: 16 }}>
187          <div className="card">
188            <h4>Invoice Details</h4>
189            <div style={{ display: "grid", gap: 12 }}>
190              <div className="form-group">
191                <label className="form-label">{(!initialData && autoCode) ? "Invoice No" : <>Invoice No
<span className="required-marker">*</span></>}</label>
192                <div className="flex gap-2">
193                  <input
194                    className="form-control"
195                    required={!autoCode}
196                    disabled={autoCode}
197                    value={invoiceNo}
198                    onChange={(e) => setInvoiceNo(e.target.value)}
199                    placeholder="e.g. INV-2026-0001"
200                  />
201                  {!initialData && (
202                    <div className="form-inline-option">
203                      <input type="checkbox" checked={autoCode} onChange={e =>
setAutoCode(e.target.checked)} id="inv_auto" />
204                      <label htmlFor="inv_auto">Auto</label>
205                    </div>
206                  )}
207                </div>
208              </div>
209
210              <div className="form-group">
211                <label className="form-label">Customer Code <span className="required-marker">*</span></
label>
212                <div style={{ display: "flex", gap: 8, alignItems: "stretch" }}>
213                  <input
214                    className="form-control"
215                    value={customerCode}
216                    onChange={(e) => setCustomerCode(e.target.value)}
217                    onBlur={handleCustomerCodeBlur}
218                    placeholder="e.g. C001"
219                    style={{ flex: 1 }}
220                  />
221                  <button
```

```
                    type="button"
                    className="btn btn-primary"
                    onClick={() => setCustomerModalOpen(true)}
                    title="List of Values"
                  >
                    LoV
                  </button>
                  {customerCode && (
                    <button
                      type="button"
                      onClick={() => { setCustomerCode(""); setCustomerDetails(null);
setCustomerLoadError(""); }}
                      title="Clear"
                      style={{
                        padding: "0 12px",
                        border: "1px solid var(--border)",
                        borderRadius: "var(--radius-sm)",
                        background: "var(--bg-body)",
                        color: "var(--text-muted)",
                        cursor: "pointer",
                        fontSize: "1.2rem",
                        lineHeight: 1,
                      }}
                    >
                      ×
                    </button>
                  )}
                </div>
                {customerLoadError && (
                  <span style={{ fontSize: "0.8rem", color: "#ef4444", marginTop: 4, display: "block" }}
>{customerLoadError}</span>
                )}
                {!customerCode && (
                  <span style={{ fontSize: "0.8rem", color: "var(--text-muted)", marginTop: 4, display:
"block" }}>Required. Type code and blur, or use LoV to select.</span>
                )}
              </div>

              <div className="form-group">
                <label className="form-label">Customer Name</label>
                <input className="form-control" disabled value={customerDetails?.name ?? ""} readOnly
placeholder="—" />
              </div>

              <div className="form-group">
                <label className="form-label">Customer Address</label>
                <input className="form-control" disabled value={customerAddressDisplay} readOnly
placeholder="—" />
              </div>

              {/* Sales Person Code: input + LoV button + clear */}
              <div className="form-group">
                <label className="form-label">Sales Person Code</label>
                <div style={{ display: "flex", gap: 8 }}>
                  <input
                    className="form-control"
                    value={salesPersonCode}
                    onChange={(e) => {
                      setSalesPersonCode(e.target.value);
                      setSalesPersonName("");
                    }}
                    placeholder="e.g. SP001"
                    style={{ flex: 1 }}
                  />
                  <button type="button" className="btn btn-primary"
                    onClick={() => setSalesPersonModalOpen(true)}>
                    LoV
                  </button>
                  {salesPersonCode && (
                    <button type="button" className="btn btn-secondary"
                      onClick={() => {
                        setSalesPersonCode("");
                        setSalesPersonName("");
                      }}
                      style={{
                        padding: "0 12px",
                        border: "1px solid var(--border)",
                        borderRadius: "var(--radius-sm)",
                        background: "var(--bg-body)",
```

```
                      color: "var(--text-muted)",
                      cursor: "pointer",
                      fontSize: "1.2rem",
                      lineHeight: 1,
                    }}>
                    ×
                  </button>
                )}
              </div>
            </div>

            <div className="form-group">
              <label className="form-label">Sales Person Name</label>
              <input className="form-control" disabled value={salesPersonName} placeholder="—" />
            </div>

            <CustomerPickerModal
              isOpen={customerModalOpen}
              onClose={() => setCustomerModalOpen(false)}
              initialSearch={customerCode}
              onSelect={(code) => {
                setCustomerCode(String(code));
                setCustomerModalOpen(false);
              }}
            />

            <SalesPersonPickerModal
              isOpen={salesPersonModalOpen}
              onClose={() => setSalesPersonModalOpen(false)}
              onSelect={(code, name) => {
                setSalesPersonCode(code);
                setSalesPersonName(name);
                setSalesPersonModalOpen(false);
              }}
            />

            <div style={{ display: "grid", gridTemplateColumns: "1fr 1fr", gap: 12 }}>
              <div className="form-group">
                <label className="form-label">Invoice Date <span className="required-marker">*</span></
label>
                <input
                  type="date"
                  className="form-control"
                  value={invoiceDate}
                  onChange={(e) => setInvoiceDate(e.target.value)}
                />
              </div>
              <div className="form-group">
                <label className="form-label">VAT Rate</label>
                <div style={{ display: 'flex', alignItems: 'center', gap: 8 }}>
                  <input
                    type="number"
                    step="1"
                    min="0"
                    max="100"
                    className="form-control"
                    value={vatPercent}
                    onChange={(e) => setVatPercent(e.target.value)}
                  />
                  <span style={{ fontSize: '0.8rem', color: 'var(--text-muted)', whiteSpace: 'nowrap' }}
>%</span>
                </div>
              </div>
            </div>
          </div>

          <div className="card invoice-summary-card" style={{ height: "fit-content" }}>
            <h4>Summary</h4>
            <div style={{ display: "grid", gap: 8 }}>
              <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
"var(--text-muted)" }}>
                <span>Total Price</span>
                <span className="amount">{submitting ? "..." : formatBaht(totalPrice)}</span>
              </div>
              <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
"var(--text-muted)" }}>
                <span>Total Discount</span>
                <span className="amount">{submitting ? "..." : formatBaht(totalDiscount)}</span>
```

```
            </div>
            <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
"var(--text-muted)" }}>
              <span>Net Price</span>
              <span className="amount">{submitting ? "..." : formatBaht(netPrice)}</span>
            </div>
            <div style={{ display: "flex", justifyContent: "space-between", fontSize: "0.875rem", color:
"var(--text-muted)" }}>
              <span>VAT ({Number(vatPercent).toFixed(0)}%)</span>
              <span className="amount">{submitting ? "..." : formatBaht(vatAmount)}</span>
            </div>
            <div style={{ borderTop: "1px solid var(--border)", paddingTop: 10, marginTop: 2, display:
"flex", justifyContent: "space-between", fontSize: "1.1rem", fontWeight: 700, color: "var(--primary)" }}>
              <span>Amount Due</span>
              <span>{submitting ? "..." : formatBaht(amountDue)}</span>
            </div>
          </div>
          <div style={{ marginTop: 16 }}>
            <button
              type="submit"
              className="btn btn-primary"
              style={{ width: "100%", padding: "10px 14px", fontSize: "0.875rem" }}
              disabled={submitting || hasEmptyProduct || hasEmptyCustomer}
            >
              {submitting ? (initialData ? "Saving..." : "Creating...") : (initialData ? "Save Changes" :
"Create Invoice")}
            </button>
            {(hasEmptyCustomer || hasEmptyProduct) && (
              <div style={{ marginTop: 6, fontSize: '0.75rem', color: '#ef4444', textAlign: 'center' }}>
                {hasEmptyCustomer && <div>Please enter customer code or select from LoV</div>}
                {hasEmptyProduct && <div>Please select a product for all items</div>}
              </div>
            )}
          </div>
        </div>
      </div>

      <LineItemsEditor value={items} onChange={setItems} />
    </form>
    </>
  );
}

```

Invoice Page (view/edit discount columns)

```
// Invoice page (View, Create & Edit)
// - /invoices/new !’ mode="create"
// - /invoices/:id !’ mode="view"
// - /invoices/:id/edit !’ mode="edit"
import React from "react";
import { useNavigate, Link, useParams } from "react-router-dom";
import { getInvoice, createInvoice, updateInvoice } from "../../api/invoices.api.js";
import { toast } from "react-toastify";
import { formatBaht, formatDate } from "../../utils.js";
import InvoiceForm from "../../components/InvoiceForm.jsx";
import Loading from "../../components/Loading.jsx";

export default function InvoicePage({ mode: propMode }) {
    const { id } = useParams();
    const mode = propMode || (id ? "view" : "create");
    const nav = useNavigate();

    const [invoiceData, setInvoiceData] = React.useState(null);
    const [initialData, setInitialData] = React.useState(null);
    const [err, setErr] = React.useState("");
    const [submitting, setSubmitting] = React.useState(false);
    const [loading, setLoading] = React.useState(true);

    React.useEffect(() => {
        if (mode === "create") {
            setLoading(false);
        } else if (mode === "view") {
            getInvoice(id)
                .then((data) => {
                    setInvoiceData(data);
                    setLoading(false);
                })
                .catch((e) => {
                    setErr(String(e.message || e));
                    setLoading(false);
                });
        } else {
            // edit mode
            getInvoice(id)
                .then((inv) => {
                    setInvoiceData(inv);

                    const h = inv.header;
                    setInitialData({
                        invoice_no: h.invoice_no,
                        customer_code: h.customer_code,
                        customer_label: `${h.customer_code || ''} - ${h.customer_name}`.replace(/^ - /,
''),
                        sales_person_code: h.sales_person_code || "",
                        sales_person_name: h.sales_person_name || "",
                        invoice_date: h.invoice_date,
                        vat_rate: h.vat_rate != null ? Number(h.vat_rate) : 0.07,
                        line_items: inv.line_items.map(li => ({
                            line_item_id: li.id,
                            product_code: li.product_code,
                            product_name: li.product_name,
                            product_label: `${li.product_code} - ${li.product_name}`,
                            units_code: li.units_code,
                            quantity: li.quantity,
                            unit_price: li.unit_price,
                            line_discount_percent: Number(li.line_discount_percent || 0),
                        }))
                    });
                    setLoading(false);
                })
                .catch((e) => {
                    setErr(String(e.message || e));
                    setLoading(false);
                });
        }
    }, [id, mode]);

    async function onSubmit(payload) {
        setErr("");
        setSubmitting(true);
        try {
            if (mode === "create") {
```

```
                const res = await createInvoice(payload);
                toast.success("Invoice created.");
                nav(`/invoices/${encodeURIComponent(res.invoice_no)}`);
            } else {
                await updateInvoice(id, payload);
                toast.success("Invoice updated.");
                nav(`/invoices/${encodeURIComponent(id)}`);
            }
        } catch (e) {
            const msg = String(e.message || e);
            setErr(msg);
            toast.error(msg);
        } finally {
            setSubmitting(false);
        }
    }

    const handlePrint = () => window.print();

    if (loading) return <Loading size="large" />;

    const isView = mode === "view";
    const isCreate = mode === "create";

    // View Mode - Invoice Preview with Print
    if (isView && invoiceData) {
        const h = invoiceData.header;
        const lines = invoiceData.line_items || [];
        const vatRate = Number(h.vat_rate || 0.07);
        const totalPrice = Number(h.total_price || h.total_amount || 0);
        const totalDiscount = Number(h.total_discount || 0);
        const netPrice = totalPrice - totalDiscount;
        const vatAmount = Number(h.vat_amount || h.vat || 0);

        return (
            <div className="invoice-preview">
                <div className="page-header no-print">
                    <h3 className="page-title">Invoice {h.invoice_no}</h3>
                    <div className="flex gap-4">
                        <Link to="/invoices" className="btn btn-outline">!• Back</Link>
                        <Link to={`/invoices/${id}/edit`} className="btn btn-outline">Edit</Link>
                        <button onClick={handlePrint} className="btn btn-primary">
                            <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><path d="M6
9V2h12v7"></path><path d="M6 18H4a2 2 0 0 1-2-2v-5a2 2 0 0 1 2-2h16a2 2 0 0 1 2 2v5a2 2 0 0 1-2 2h-2"></
path><rect x="6" y="14" width="12" height="8"></rect></svg>
                            Print PDF
                        </button>
                    </div>
                </div>

                <div className="card">
                    <div className="flex justify-between mb-4">
                        <div>
                            <div className="brand mb-4">InvoiceDoc v2</div>
                            <div className="font-bold">Customer</div>
                            <div>{h.customer_name}</div>
                            <div className="text-muted">{h.address_line1 || "-"}</div>
                            <div className="text-muted">{h.address_line2 || ""}</div>
                            <div className="text-muted">{h.country_name || "-"}</div>
                        </div>
                        <div className="text-right">
                            <h2 className="mb-4">INVOICE</h2>
                            <div><span className="font-bold">Date:</span> {formatDate(h.invoice_date)}</
div>
                            <div><span className="font-bold">Invoice No:</span> {h.invoice_no}</div>
                            {h.sales_person_code && (
                              <div>
                                <span className="font-bold">Sales Person:</span> {h.sales_person_code} –
{h.sales_person_name}
                              </div>
                            )}
                        </div>
                    </div>

                    <div className="table-container">
                        <table className="modern-table">
                            <thead>
                                <tr>
```

```
                                    <th>Product</th>
                                    <th>Unit</th>
                                    <th className="text-right">Qty</th>
                                    <th className="text-right">Unit Price</th>
                                    <th className="text-right">Extended</th>
                                    <th className="text-right">Disc %</th>
                                    <th className="text-right">Disc Amt</th>
                                    <th className="text-right">Net Price</th>
                                </tr>
                            </thead>
                            <tbody>
                                {lines.map((li) => (
                                    <tr key={li.id}>
                                        <td>{li.product_code} - {li.product_name}</td>
                                        <td>{li.units_code}</td>
                                        <td className="text-right">{Number(li.quantity || 0).toFixed(2)}</
td>
                                        <td className="text-right">{formatBaht(li.unit_price)}</td>
                                        <td className="text-right">{formatBaht(li.extended_price)}</td>
                                        <td className="text-right">{Number(li.line_discount_percent ||
0).toFixed(2)}%</td>
                                        <td className="text-right">{formatBaht(li.line_discount_amount)}</
td>
                                        <td className="text-right font-
bold">{formatBaht(li.line_net_price)}</td>
                                    </tr>
                                ))}
                            </tbody>
                        </table>
                    </div>

                    <div className="mt-4 flex justify-between">
                        <div className="no-print text-muted" style={{ maxWidth: 300, fontSize: '0.8rem' }}>
                            Thank you for your business. Please pay within 30 days.
                        </div>
                        <div style={{ minWidth: 240 }}>
                            <div className="flex justify-between mb-2">
                                <span>Total Price:</span>
                                <span>{formatBaht(totalPrice)}</span>
                            </div>
                            <div className="flex justify-between mb-2">
                                <span>Total Discount:</span>
                                <span>{formatBaht(totalDiscount)}</span>
                            </div>
                            <div className="flex justify-between mb-2">
                                <span>Net Price:</span>
                                <span>{formatBaht(netPrice)}</span>
                            </div>
                            <div className="flex justify-between mb-2">
                                <span>VAT ({(vatRate * 100).toFixed(0)}%):</span>
                                <span>{formatBaht(vatAmount)}</span>
                            </div>
                            <div className="flex justify-between mt-4 p-2 bg-body font-bold"
style={{ fontSize: '1.1rem' }}>
                                <span>Total Due:</span>
                                <span>{formatBaht(h.amount_due)}</span>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        );
    }

    // Create/Edit Mode - Form
    const title = isCreate ? "Create Invoice" : `Edit Invoice ${id}`;

    return (
        <div className="invoice-page">
            <div className="page-header">
                <h3 className="page-title">{title}</h3>
                <Link to="/invoices" className="btn btn-outline">
                    <svg style={{ marginRight: 8 }} width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round"><line x1="19" y1="12" x2="5"
y2="12"></line><polyline points="12 19 5 12 12 5"></polyline></svg>
                    Back
                </Link>
            </div>
            {err && <div className="alert alert-error">{err}</div>}
```

```
            <InvoiceForm
                onSubmit={onSubmit}
                submitting={submitting}
                initialData={isCreate ? null : initialData}
            />
        </div>
    );
}

```